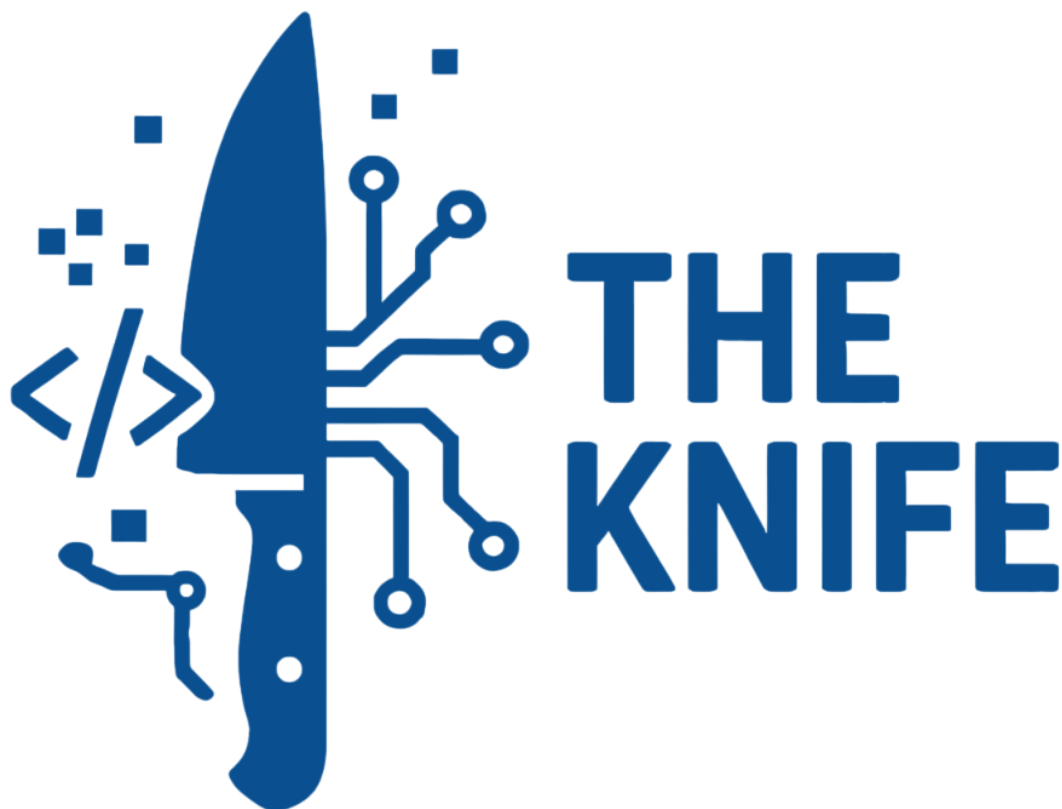




Manuale Tecnico

Versione 2.0

Università degli Studi dell'Insubria - Laurea Triennale in Informatica
Laboratorio Interdisciplinare B: Progetto The Knife

Sviluppato da:

GIACOMO TANCREDI – Matricola 760741

GAIA PELLEGRINI – Matricola 759909

ELENA CATELLI – Matricola 763070

CAMILLA RIZZI – Matricola 761630

Indice

1. Introduzione.....	4
1.1 Premessa.....	4
2. Installazione e configurazione.....	4
2.1 Requisiti minimi:.....	4
2.2 Installazione.....	5
2.3 Configurazione.....	5
3. Software Design.....	6
3.1 Struttura dell'applicazione.....	6
3.2. Connessione al database.....	10
3.2.1 Parametri di configurazione.....	10
3.2.2 Creazione della connessione.....	10
3.2.3 Considerazioni.....	11
3.3 Traduzione delle entità relazionali in entità Java.....	11
3.3.1 Struttura generale.....	11
3.3.2 Validazione dei dati.....	12
3.3.3 Considerazioni.....	12
4 Architettura.....	13
4.1 Obiettivi e vincoli.....	13
4.2 Descrizione dei livelli.....	14
4.3 Comunicazione client-server.....	15
4.4 Gestione della concorrenza.....	16
4.5 Librerie utilizzate.....	16
4.5.1 Librerie standard utilizzate.....	17
4.6 Cifratura dei dati.....	19
4.7 Decisioni Chiave e Alternative.....	19
4.7.1 Persistenza dei dati.....	19
4.7.2 Interfaccia utente.....	20
4.7.3 Comunicazione client-server.....	20
4.7.4 Gestione della concorrenza.....	20
5. Gestione e Persistenza dei Dati.....	21
5.1 Schema del Database.....	21
5.1.1 Tabella utenti.....	22
5.1.2 Tabella TipiCucina.....	22
5.1.3 Tabella Ristorante.....	22

5.1.4 Tabella Preferiti.....	22
5.1.5 Tabella Recensioni.....	23
5.2 Struttura interna DAO.....	23
5.3 Connessione al database.....	23
5.5 Indici.....	24
5.6 Query SQL.....	24
5.6.1 Utente.....	25
5.6.2 Ristorante.....	26
5.6.3 TipiCucina.....	27
5.6.4 Preferiti.....	27
5.6.5 Recensioni.....	28
5.7 Schema relazionale.....	29
6. Scelte algoritmiche.....	30
6.1 Ricerca dei ristoranti.....	30
6.2 Gestione degli utenti e autenticazione.....	30
6.3 Inserimento e gestione dei ristoranti.....	31
6.4 Recensioni e punteggi.....	31
7. Ulteriori Diagrammi UML.....	32
7.1 Diagrammi delle classi – modulo server.....	32
7.2 Diagrammi di sequenza.....	34
7.3 Diagrammi di stato.....	35
7.4 Diagrammi delle classi – modulo client.....	35
7.5 Diagramma delle classi – modulo shared.....	37
8. Bibliografia e Sitografia.....	38

1. Introduzione

“*The Knife*” è un progetto realizzato nell’ambito del corso di *Laboratorio Interdisciplinare B*, parte integrante del percorso formativo del corso di laurea triennale in Informatica presso l’Università degli Studi dell’Insubria.

Il progetto costituisce la seconda e fase conclusiva di sviluppo dell’applicazione, sviluppato in precedenza nell’ambito del *Laboratorio Interdisciplinare A* come applicazione a riga di comando con persistenza su file CSV.

In questa fase, il progetto è stato riprogettato come applicazione client-server, con persistenza dei dati sul database relazionale *PostgreSQL* e interfaccia grafica sviluppata in *Java Swing*.

Il progetto è stato interamente sviluppato in *Java 23*, con particolare attenzione alla chiarezza, modularità e manutenibilità del codice.

Le fasi di sviluppo e testing sono state condotte su sistemi operativi *Windows 10* e *Windows 11*, assicurandone il corretto funzionamento in tali ambienti.

Per lo sviluppo è stato utilizzato l’ambiente di sviluppo integrato *IntelliJ IDEA*, a supporto di una progettazione efficiente e strutturata.

1.1 Premessa

Notasi che in questo manuale non verrà discusso il funzionamento di tutte le classi e i metodi contenuti al loro interno: verranno menzionati i componenti cruciali al funzionamento del programma e quelli di maggior rilevanza e/o complessità. Per un elenco completo e dettagliato, fare riferimento alla Javadoc del progetto

Si segnala, inoltre, che non tutti i diagrammi UML prodotti sono inclusi in questo manuale. Essi sono allegati alla documentazione tecnica e possono essere consultati nella cartella dedicata

Per ulteriori informazioni sul funzionamento del programma, incluse le istruzioni per l’avvio, consultare il *Manuale Utente*.

2. Installazione e configurazione

2.1 Requisiti minimi:

Per il corretto funzionamento dell’applicazione è necessario disporre dei seguenti requisiti:

- *Sistema Operativo*: Windows 10 o superiore
- *Java*: JDK/JRE 23 o superiore
- *PostgreSQL*: versione 18 o superiore

Nota: l’applicazione è stata sviluppata e testata sui sistemi operativi Windows 10 e Windows 11, per tanto non si assicura che sia funzionale su MacOS e Linux.

2.2 Installazione

Per installare ed eseguire l'applicazione, procedere come segue:

1. Verificare che *Java JDK/JRE 23* (o una versione successiva) sia installato sul sistema eseguendo il comando: `java -version`
 - Qualora Java non fosse installato, è possibile scaricarlo dal sito ufficiale di Oracle: <https://www.oracle.com/java/technologies/javase/jdk23-archive-downloads.html>
2. Installare *PostgreSQL* e assicurarsi di avere il servizio attivo
 - Disponibile al seguente link: <https://www.postgresql.org/download/>
3. Creare un database denominato “TheKnife” ed eseguire lo script SQL fornito con il progetto per creare le tabelle necessarie o caricare il database fornito.
4. Scaricare i file eseguibili dell'applicazione:
 - `TheKnifeServer.jar`
 - `TheKnifeClient.jar`
5. Configurare le variabili d'ambiente necessarie per la connessione al database:
 - `DB_URL`
 - `$env:DB_URL="jdbc:postgresql://localhost:5432/postgres"`
 - `DB_USER`
 - `$env:DB_USER="postgres"`
 - `DB_PASSWORD`
 - `$env:DB_PASSWORD="la_tua_password"`
6. Verificare che tutte le librerie richieste siano presenti nel *classpath* dell'applicazione.
7. Avviare il server mediante il comando: `java --enable-preview -jar bin\theknife-1.1-server.jar`
8. Successivamente, avviare il client con il comando: `java --enable-preview -jar bin\theknife-1.1-client.jar` o con doppio clic sull'eseguibile

2.3 Configurazione

L'applicazione richiede esclusivamente la configurazione della connessione al database PostgreSQL tramite le variabili d'ambiente `DB_URL`, `DB_USER` e `DB_PASSWORD`. Per maggiori informazioni leggere il readme.

Una volta configurate tali variabili e avviato il database, il server stabilisce automaticamente la connessione e l'applicazione risulta immediatamente operativa.

Tutti i dati relativi agli utenti, ai ristoranti e alle recensioni vengono memorizzati in modo persistente all'interno del database PostgreSQL.

3. Software Design

Per lo sviluppo dell'applicazione è stata adottata *Java 23*, una versione recente del linguaggio che introduce numerose funzionalità volte a semplificare lo sviluppo del software, riducendo la quantità di codice boilerplate e migliorandone la leggibilità, la manutenibilità e la robustezza.

L'applicazione è stata progettata e sviluppata utilizzando *Java 23* ed è stata testata sui sistemi operativi *Windows 10* e *Windows 11*, garantendone il corretto funzionamento in tali ambienti.

Per la gestione e la persistenza dei dati è stato impiegato il DBMS *PostgreSQL*, in conformità con i requisiti definiti dai committenti. Il database è utilizzato per la memorizzazione permanente delle informazioni relative agli utenti, ai ristoranti, alle recensioni e a tutti gli altri dati gestiti dall'applicazione.

La comunicazione tra client e server è stata implementata mediante *socket TCP*, che consentono uno scambio di dati affidabile tra le due componenti dell'applicazione. Attraverso questo meccanismo il client invia le proprie richieste al server, il quale le elabora e restituisce le relative risposte, garantendo una comunicazione bidirezionale (*full-duplex*) e persistente per l'intera durata della connessione.

L'interfaccia grafica del client è stata sviluppata utilizzando *Java Swing*, toolkit grafico della piattaforma Java, scelto per la realizzazione di un'interfaccia desktop intuitiva, modulare e facilmente manutenibile.

Le funzionalità offerte dall'applicazione sono rappresentate mediante lo *Use Case Diagram* (Figura 1), che fornisce una visione d'insieme delle interazioni tra gli attori del sistema e dei casi d'uso disponibili. Tali funzionalità sono state descritte dal punto di vista dell'utilizzatore nel *Manuale Utente* e vengono approfondite nei paragrafi successivi del presente documento (vedi sezione).

3.1 Struttura dell'applicazione

I committenti hanno richiesto la realizzazione di un'applicazione client-server in Java. Per soddisfare questa esigenza, il sistema è stato suddiviso in tre moduli distinti:

- server: contenente il server;
- client: contenente il client;
- shared: contenente le entità (classi) condivise tra server e client

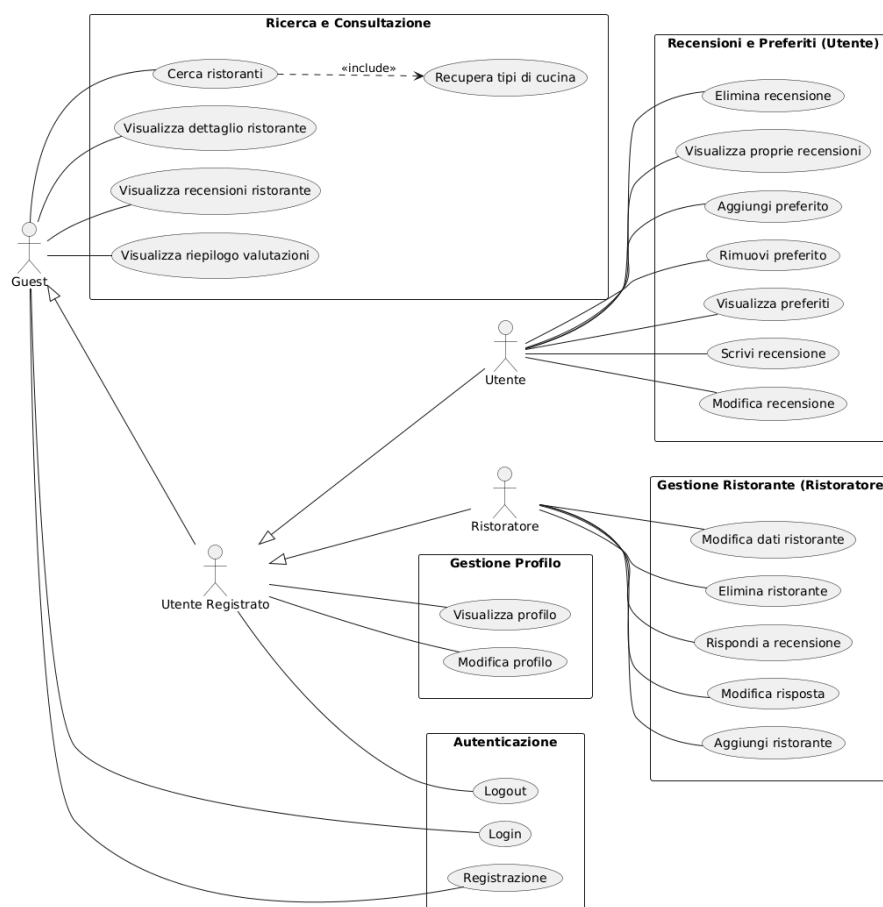
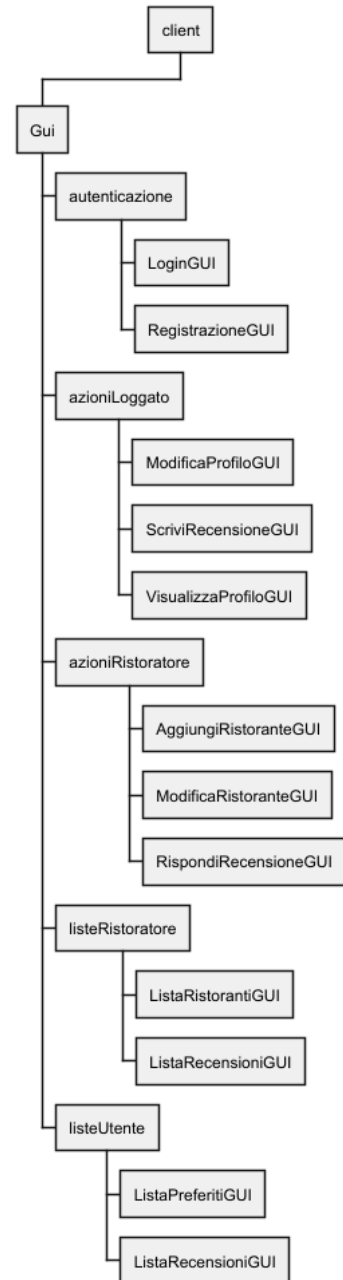
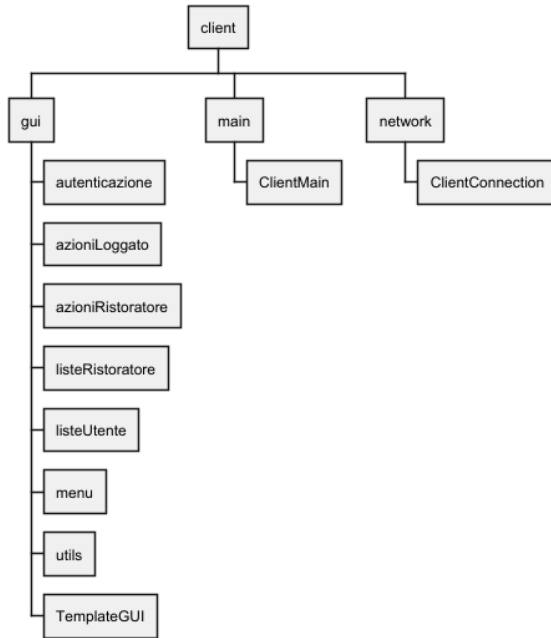


Figura 1: Use Case Class Diagram The Knife

Come sistema di build è stato utilizzato *Apache Maven*, che consente di automatizzare la compilazione del progetto e la gestione delle dipendenze. Ciascun modulo dell'applicazione dispone di un proprio file `pom.xml`, nel quale sono definite le librerie necessarie al corretto funzionamento del modulo stesso.

L'organizzazione del codice sorgente è stata realizzata suddividendo i package in base alle rispettive funzionalità, con l'obiettivo di favorire la modularità, la manutenibilità e la separazione delle responsabilità tra i diversi componenti del sistema.

La *Figura 2* riporta la struttura dei package del progetto, evidenziandone l'organizzazione e le relazioni tra i principali moduli dell'applicazione.



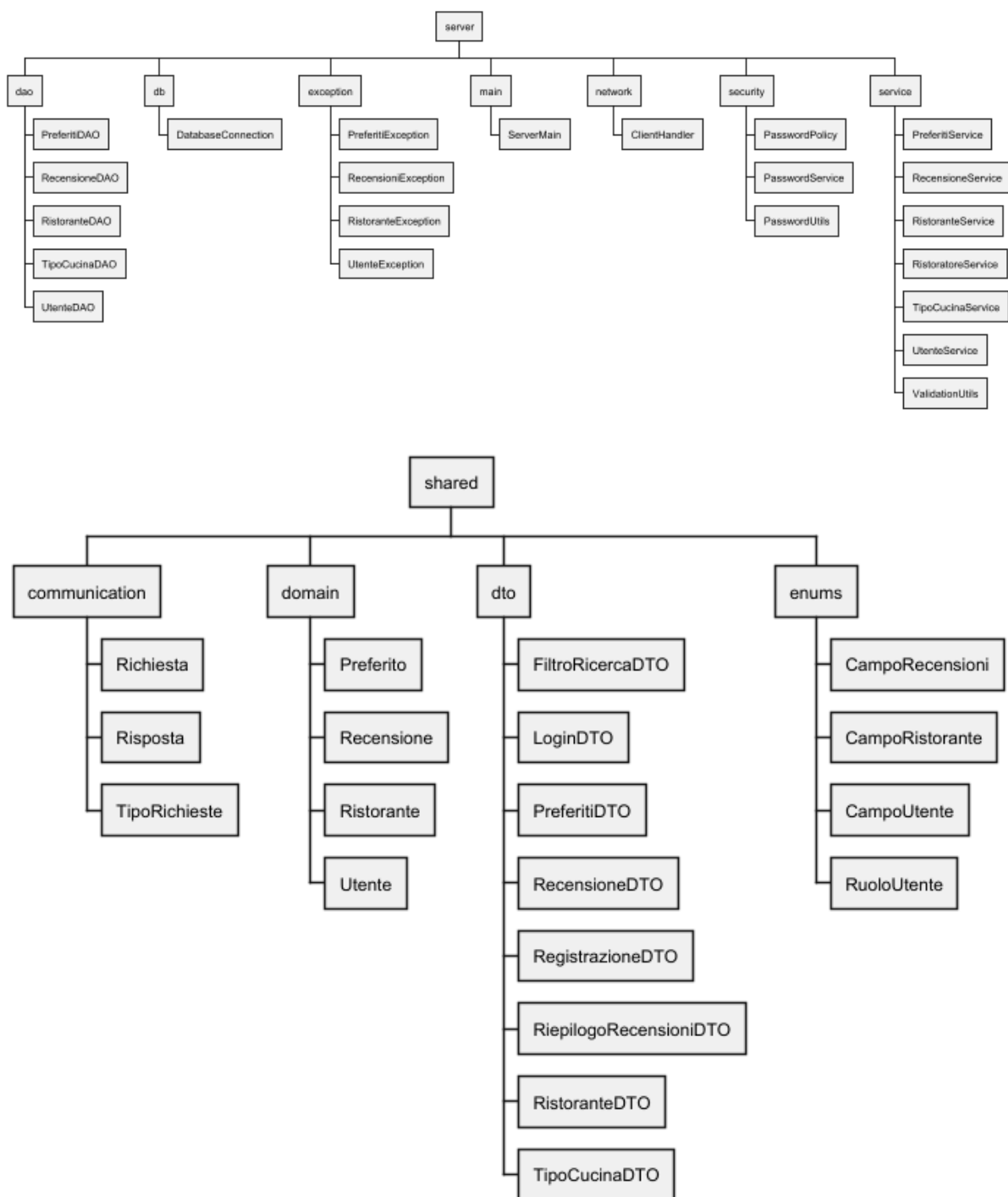


Figura 2: Struttura moduli: client, server, shared.

3.2. Connessione al database

La gestione della connessione al database è centralizzata nella classe `DatabaseConnection`, collocata nel package `main.java.server.db`. Questa classe implementa il pattern *utility class*: il costruttore è privato per impedirne l'istanziamento, e l'accesso alle funzionalità avviene esclusivamente tramite il metodo statico `getConnection()`.

3.2.1 Parametri di configurazione

I parametri necessari per stabilire la connessione (URL del database, utente e password) sono letti da variabili d'ambiente, con valori di default utilizzati in loro assenza:

Parametro	Variabile d'ambiente	Valore di default
URL	DB_URL	<code>jdbc:postgresql://localhost:5432/postgres</code>
Utente	DB_USER	<code>postgres</code>
Password	DB_PASSWORD	la password inizializzata dall'utente

Questo approccio, basato su `System.getenv().getOrDefault(...)`, consente di adattare la configurazione ai diversi ambienti di esecuzione (sviluppo, test, produzione) senza modificare il codice sorgente, favorendo la portabilità dell'applicazione e la separazione tra codice e configurazione, in linea con i principi delle *twelve-factor app*.

3.2.2 Creazione della connessione

Il metodo `getConnection()` restituisce un oggetto `Connection` invocando `DriverManager.getConnection(URL, USER, PASSWORD)`. Il driver JDBC per PostgreSQL viene individuato automaticamente dal `DriverManager` grazie al meccanismo di service discovery (`java.sql.Driver`), a condizione che la relativa libreria (`postgresql-jdbc`) sia presente nel classpath del progetto.

```
public class DatabaseConnection {
    private static final String URL = System.getenv().getOrDefault(
        "DB_URL", "jdbc:postgresql://localhost:5432/postgres"
    );
    private static final String USER = System.getenv().getOrDefault(
        "DB_USER", "postgres"
    );
    private static final String PASSWORD = System.getenv().getOrDefault(
        "DB_PASSWORD", "postgres"
    );
    private DatabaseConnection() { }
    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

Il metodo dichiara `throws SQLException`, delegando quindi al chiamante la responsabilità di gestire eventuali errori di connessione (ad esempio credenziali errate, database non raggiungibile o servizio non attivo).

3.2.3 Considerazioni

Va evidenziato che, allo stato attuale, la classe non implementa un connection pool: ogni chiamata a `getConnection()` apre una nuova connessione fisica al database. In un contesto di produzione con carichi di lavoro concorrenti, sarebbe opportuno valutare l'adozione di un pool di connessioni per ottimizzare le prestazioni e la gestione delle risorse.

3.3 Traduzione delle entità relazionali in entità Java

Prima di illustrare l'architettura del sistema, è utile chiarire il processo di traduzione delle entità relazionali in entità Java.

Le principali entità sono:

- Preferito;
- Recensione;
- Ristorante;
- Utente;
- TipiCucina;

Queste corrispondono alle tabelle del database `preferiti`, `recensioni`, `ristoranti`, `utenti` e `tipiCucina`.

Lato applicativo, ciascuna entità del dominio è stata implementata tramite una classe POJO, collocata nel package `main.java.shared.domain`. La traduzione degli attributi delle tabelle SQL nei rispettivi campi Java è stata sostanzialmente 1:1, mantenendo una corrispondenza diretta tra lo schema del database e il modello applicativo. Le entità individuate sono `Utente`, `Ristorante`, `Recensione` e `Preferito`; in Figura 3 è presente il class diagram completo.

3.3.1 Struttura generale

Ogni classe di dominio implementa l'interfaccia `Serializable`, per consentirne la trasmissione tra i diversi livelli dell'applicazione (ad esempio tra client e server tramite socket, o in fase di persistenza in sessione). Tutte seguono uno schema strutturale comune:

- *Attributi*: dichiarati privati, riflettono in maniera diretta le colonne delle rispettive tabelle del database;
- *Costruttori*: è generalmente previsto un costruttore vuoto, necessario al corretto funzionamento del DAO in fase di istanziazione tramite reflection, e un costruttore parametrico per la creazione diretta dell'oggetto a partire dai valori noti. Nei costruttori parametrici, i campi di tipo `String` vengono tipicamente normalizzati tramite `trim()` (e, in alcuni casi, `toLowerCase()`, come per l'email e la città in `Utente`) per garantire coerenza nei dati memorizzati;

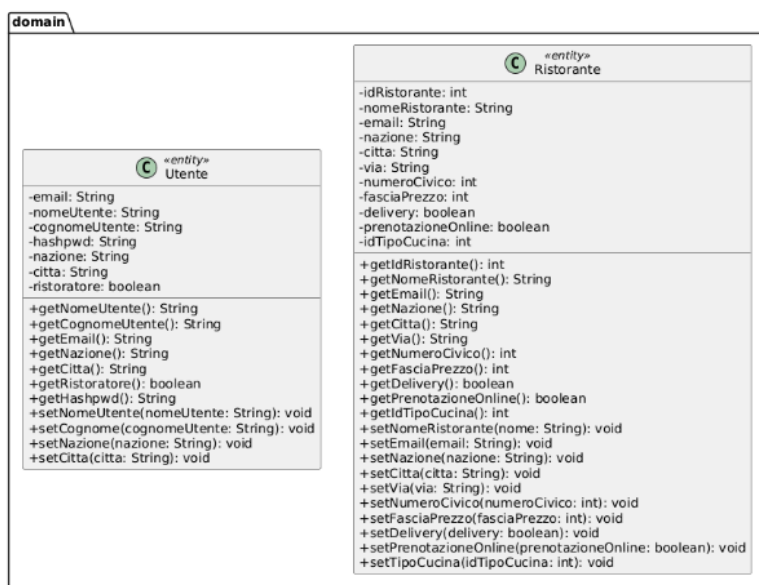
- *Metodi get/set*: ciascun attributo è dotato dei corrispondenti metodi di accesso, mentre i metodi di modifica sono presenti selettivamente sui campi per cui è prevista una variazione nel tempo (ad esempio i dati anagrafici di un ristorante o di un utente, oppure il testo di una recensione);
- *Metodo toString()*: ridefinito in ciascuna classe per restituire una rappresentazione testuale sintetica dell'oggetto, con i campi concatenati tramite un separatore (virgola o asterisco, a seconda dell'entità), utile in fase di debug, logging e serializzazione semplificata.

3.3.2 Validazione dei dati

In alcuni casi, i metodi set incorporano logica di validazione a garanzia dell'integrità del dato. Ad esempio, il metodo `setValutazione()` della classe `Recensione` verifica che il punteggio assegnato sia compreso nell'intervallo consentito (1–5), sollevando un'eccezione applicativa dedicata (`RecensioniException`) in caso contrario. Questo approccio consente di centralizzare il controllo di coerenza direttamente a livello di modello, evitando la propagazione di dati non validi verso il livello di persistenza.

3.3.3 Considerazioni

L'adozione uniforme del pattern POJO per tutte le entità di dominio garantisce coerenza e prevedibilità nella gestione dei dati a livello applicativo, semplificando l'integrazione con il livello di persistenza (DAO) e con la logica di trasmissione dati tra client e server. La presenza di costruttori vuoti e di setter mutabili, in luogo di un'implementazione basata su Record immutabili, risulta funzionale alle esigenze di reflection e ai meccanismi di aggiornamento incrementale dei dati richiesti dal DAO.



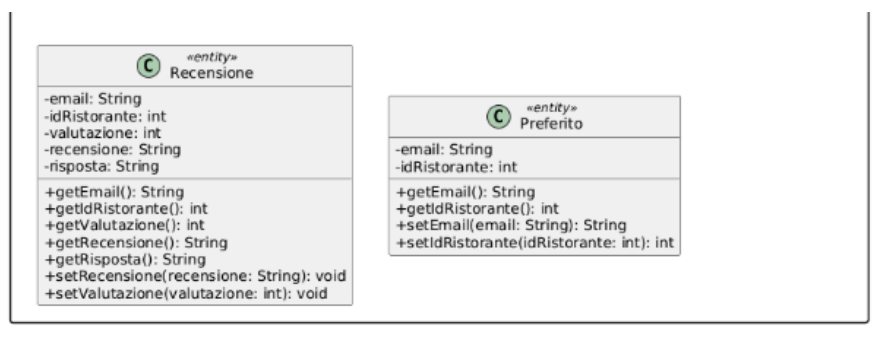


Figura 3: Entità POJO.

Tutte le entità descritte implementano l'interfaccia `Serializable` e sono collocate nel package `main.java.shared.domain`, poiché devono essere costantemente scambiate tra client e server tramite i socket TCP.

A differenza di altri contesti in cui un'associazione multi-a-molti tra due entità viene gestita esclusivamente a livello di tabella di collegamento, senza una diretta rappresentazione come classe Java, nel caso della relazione tra `Utente` e `Ristorante` si è scelto di mappare esplicitamente l'associazione tramite l'entità `Preferito`.

Questa scelta è motivata dal fatto che tale relazione (un utente può aggiungere ai preferiti un numero indefinito di ristoranti, un ristorante può essere preferito da un numero indefinito di utenti) necessita di essere manipolata direttamente lato applicativo — in fase di aggiunta, rimozione e consultazione dei preferiti — e non si limita a un semplice collegamento di supporto interno al database.

La classe `Preferito`, contenendo i soli identificativi delle due entità coinvolte (`email` e `idRistorante`), rappresenta quindi il corrispettivo Java della tabella di collegamento, mappata 1:1 secondo lo stesso criterio adottato per le altre entità di dominio.

4 Architettura

Nell'applicazione è stata adottata una classica architettura a tre livelli. In Figura 4 ne è riportata una rappresentazione grafica.

4.1 Obiettivi e vincoli

L'obiettivo principale del progetto è la realizzazione di “*The Knife*”, un'applicazione didattica ispirata alla piattaforma “*TheFork*”, in grado di simulare le principali funzionalità di ricerca, consultazione e gestione dei ristoranti.

Durante la progettazione è stata posta particolare attenzione alla qualità del software, privilegiando un'architettura modulare che favorisse la leggibilità del codice, la separazione delle responsabilità e la manutenibilità del sistema. Un ulteriore obiettivo è stato quello di

realizzare un'interfaccia semplice e intuitiva, in grado di garantire un'esperienza d'uso efficace sia agli utenti *Cliente*, interessati alla ricerca e alla consultazione dei ristoranti, sia agli utenti *Ristoratore*, responsabili della gestione delle informazioni relative ai propri locali.

Lo sviluppo del progetto è stato inoltre guidato da alcuni vincoli progettuali. L'applicazione è stata implementata in *Java* secondo un'architettura *client-server*, separando la logica di business e il livello di persistenza dall'interfaccia utente. La persistenza dei dati è stata affidata al DBMS *PostgreSQL*, mentre l'interfaccia grafica è stata sviluppata mediante *Java Swing*. La comunicazione tra client e server è stata realizzata tramite *socket TCP*, adottando un modello concorrente lato server per consentire la gestione simultanea di più connessioni client.

4.2 Descrizione dei livelli

Dal basso verso l'alto, l'architettura è composta dai seguenti livelli:

- *Persistence Layer (Database)*: il livello di persistenza dei dati, implementato utilizzando il database relazionale *PostgreSQL*.
- *Data Access Layer*: questo livello fornisce un'interfaccia di accesso ai dati, astratta tramite il pattern strutturale *Data Access Object (DAO)*, che permette di isolare la logica di accesso ai dati dalle altre parti del sistema.
- *Service Layer*: qui viene implementata la logica di business dell'applicazione. Nella fase iniziale del ciclo di vita dell'applicativo il Service Layer si occupa principalmente della validazione dei dati provenienti dal client e del logging delle eccezioni provenienti dal *Data Access Layer*.
- *Presentation Layer*: corrisponde all'interfaccia grafica utente (*Graphical User Interface, GUI*) e alla logica di presentazione, che gestisce l'interazione tra l'utente e il sistema.

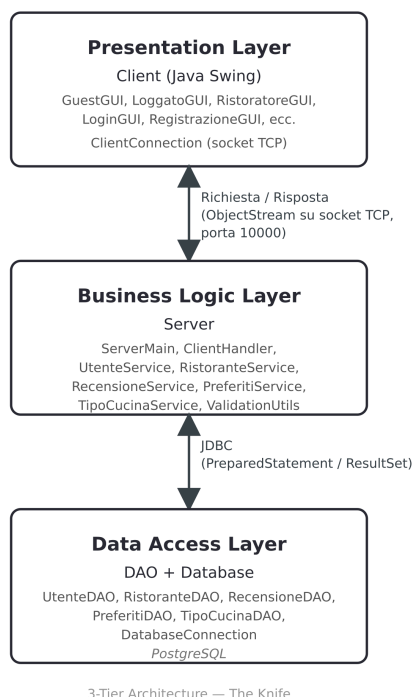


Figura 4: 3-Tier Architecture.

Ad eccezione del livello di presentazione, ciascun livello dell'architettura espone un insieme di interfacce utilizzate esclusivamente dal livello immediatamente superiore. In questo modo, ogni livello delega al livello sottostante le funzionalità di cui necessita, comunicando attraverso interfacce ben definite e indipendenti dalle rispettive implementazioni.

Tale organizzazione garantisce una chiara separazione delle responsabilità (*Separation of Concerns*), riducendo l'accoppiamento tra i diversi componenti del sistema e favorendo una maggiore modularità. Di conseguenza, le modifiche apportate a un determinato livello hanno un impatto limitato sugli altri livelli, semplificando le attività di manutenzione, evoluzione e testing dell'applicazione.

4.3 Comunicazione client-server

La comunicazione tra client e server è realizzata mediante *socket TCP*, utilizzando la porta *10000*. Lo scambio delle informazioni avviene attraverso la serializzazione di oggetti Java, permettendo il trasferimento strutturato dei dati tra le due componenti dell'applicazione.

Ogni richiesta inviata dal client è rappresentata dalla classe *Richiesta*, costituita da:

- Il tipo di operazione da eseguire, identificato mediante l'enumerazione *TipoRichieste*, che definisce i 27 tipi di operazioni supportate dal sistema;
- Il contenuto della richiesta, rappresentato da un opportuno *Data Transfer Object (DTO)* contenente i dati necessari all'esecuzione dell'operazione.

Il server elabora la richiesta e restituisce un oggetto di tipo *Risposta*, contenente:

- L'esito dell'operazione;
- L'eventuale contenuto restituito;
- Un messaggio descrittivo.

Le classi *Richiesta* e *Risposta*, insieme ai DTO e alle entità di dominio condivise, sono collocate nel package *shared*, comune sia al client sia al server. Questa scelta progettuale consente di mantenere un modello dati condiviso, evitando duplicazioni di codice e garantendo la coerenza nella rappresentazione delle informazioni scambiate.

4.4 Gestione della concorrenza

Il server, avviato dalla classe *ServerMain*, rimane in ascolto sulla porta *10000* in attesa di nuove connessioni. Per ogni client che stabilisce una connessione viene creato un nuovo thread dedicato, rappresentato dalla classe *ClientHandler*.

Ciascun *ClientHandler* gestisce in modo autonomo tutte le richieste provenienti dal client associato, eseguendo un ciclo continuo di ricezione, elaborazione e invio delle risposte fino alla chiusura della connessione.

L'implementazione adottata non prevede l'utilizzo di un *thread pool*: ogni client mantiene un thread dedicato per l'intera durata della propria sessione. Sebbene questa soluzione sia meno scalabile rispetto all'impiego di un pool di thread, risulta adeguata per gli obiettivi didattici del progetto, offrendo una struttura semplice, facilmente comprensibile e di agevole manutenzione.

4.5 Librerie utilizzate

Il progetto “*The Knife*” è stato sviluppato facendo principalmente affidamento sulle librerie standard messe a disposizione dalla piattaforma Java. L'utilizzo di dipendenze esterne è stato limitato ai soli casi in cui risultavano strettamente necessarie, privilegiando così una soluzione semplice, facilmente distribuibile e di agevole manutenzione.

Questa scelta progettuale offre diversi vantaggi. Innanzitutto, garantisce un'elevata *portabilità*, consentendo l'esecuzione dell'applicazione su qualsiasi sistema dotato di una Java Virtual Machine (JVM) compatibile. Inoltre, il numero ridotto di librerie esterne semplifica la distribuzione del software e limita il rischio di incompatibilità o conflitti tra versioni differenti delle dipendenze.

Per quanto riguarda la sicurezza, si è scelto di utilizzare una libreria consolidata per la gestione delle password anziché implementare autonomamente l'algoritmo di hashing. Tale scelta riduce il rischio di errori nello sviluppo di una componente particolarmente critica e consente di adottare una soluzione ampiamente utilizzata e verificata dalla comunità.

Le principali librerie esterne impiegate nel progetto sono le seguenti:

- **PostgreSQL JDBC Driver:** driver JDBC utilizzato per consentire la comunicazione tra l'applicazione Java e il database PostgreSQL. Tale libreria implementa le specifiche JDBC necessarie per l'esecuzione delle operazioni di connessione, interrogazione e aggiornamento del database.
- **jBCrypt** (`org.mindrot.jbcrypt`): libreria che implementa l'algoritmo di hashing BCrypt, utilizzata per la memorizzazione sicura delle password degli utenti. L'impiego di BCrypt permette di proteggere le credenziali attraverso un algoritmo progettato specificamente per resistere agli attacchi di forza bruta e alle tecniche di cracking delle password.

Libreria	Package/Classe	Funzione principale
<i>PostgreSQL JDBC Driver</i>	(caricato via <code>DriverManager</code> , nessun import esplicito)	driver di connessione al database PostgreSQL
<i>jBCrypt</i>	<code>org.mindrot.jbcrypt.BCrypt</code>	hashing e verifica sicura delle password con l'algoritmo BCrypt

4.5.1 Librerie standard utilizzate

Oltre alle dipendenze esterne precedentemente descritte, il progetto fa ampio utilizzo delle librerie standard della piattaforma Java, sfruttando le funzionalità per la realizzazione delle diverse componenti dell'applicazione.

Le principali librerie impiegate sono le seguenti:

- `java.util` – utilizzata per la gestione delle strutture dati (ad esempio `ArrayList`, `List` e `Optional`), delle espressioni regolari, delle collezioni e delle funzionalità di programmazione funzionale introdotte nelle versioni più recenti di Java.
- `java.io` – impiegata per le operazioni di input/output e, in particolare, per la serializzazione e deserializzazione degli oggetti scambiati tra client e server attraverso la rete.
- `java.net` – fornisce le classi necessarie alla comunicazione di rete mediante socket TCP, consentendo la realizzazione dell'architettura client-server.
- `java.nio` – utilizzata per la gestione delle codifiche dei caratteri (charset) e per alcune operazioni di input/output più efficienti rispetto alle API tradizionali.
- `java.security` – impiegata per l'utilizzo di algoritmi crittografici, in particolare per il calcolo degli hash mediante l'algoritmo *SHA-256*, utilizzato nelle funzionalità di sicurezza dell'applicazione.
- `java.sql` – fornisce le API JDBC utilizzate per la connessione al database PostgreSQL e per l'esecuzione delle operazioni di interrogazione e aggiornamento dei dati.

- `java.awt` e `javax.swing` – utilizzate per la realizzazione dell'interfaccia grafica desktop del client, comprendendo componenti grafici, gestione degli eventi, layout e rendering dell'interfaccia utente.

Pacchetto	Classe/Interfaccia	Funzione principale
<i>java.util</i>	<code>ArrayList, List</code>	gestione di strutture dati dinamiche
	<code>Optional</code>	gestione di valori potenzialmente assenti
	<code>regex.Pattern</code>	validazione di campi tramite espressioni regolari
	<code>function.Consumer</code>	gestione di callback funzionali nella GUI
<i>java.io</i>	<code>ObjectInputStream, ObjectOutputStream</code>	serializzazione/deserializzazione degli oggetti Richiesta/Risposta scambiati tra client e server
	<code>IOException, EOFException</code>	gestione delle eccezioni di I/O sulla connessione di rete
	<code>Serializable</code>	marcatura delle classi condivise (DTO, entità) come serializzabili
<i>java.net</i>	<code>Socket, ServerSocket</code>	comunicazione di rete tra client e server tramite socket TCP
<i>java.nio.charset</i>	<code>StandardCharsets</code>	definizione della codifica dei caratteri (hashing SHA-256)
<i>java.security</i>	<code>MessageDigest</code>	calcolo di hash crittografici (algoritmo SHA-256)
	<code>NoSuchAlgorithmException</code>	Eccezione per algoritmi di sicurezza non disponibili
<i>java.sql</i>	<code>Connection, DriverManager</code>	apertura e gestione della connessione al database
	<code>PreparedStatement</code>	esecuzione di query parametriche verso il database
	<code>ResultSet</code>	lettura dei risultati restituiti dalle

		query
	SQLException	gestione delle eccezioni di accesso al database
<i>java.awt</i>	FlowLayout, Graphics	gestione del layout e del rendering nei componenti grafici
<i>java.awt.event</i>	WindowEvent, WindowAdapter	gestione degli eventi delle finestre della GUI
<i>javax.swing</i>	JFrame, JPanel, JButton, JTextField, JOptionPane, ecc.	realizzazione dell'interfaccia grafica del client
<i>javax.swing.border</i>	Border, LineBorder	personalizzazione grafica dei bordi dei componenti

4.6 Cifratura dei dati

Per garantire la sicurezza delle credenziali degli utenti, le password non vengono mai memorizzate in chiaro all'interno del database, ma sono cifrate mediante l'algoritmo di hashing *BCrypt*, utilizzando un *fattore di costo pari a 12* e la libreria *jBCrypt*.

La gestione delle password è affidata alla classe `PasswordService`, che riconosce automaticamente il formato dell'hash memorizzato nel database. In particolare, il sistema è compatibile sia con gli hash *BCrypt*, identificabili dai prefissi `$2a$`, `$2b$` e `$2y$`, sia con gli hash *SHA-256*, impiegati nelle versioni precedenti dell'applicazione.

Per garantire la retrocompatibilità senza richiedere alcun intervento da parte degli utenti, durante la procedura di autenticazione un'eventuale password memorizzata mediante *SHA-256* viene verificata e successivamente ricalcolata utilizzando *BCrypt*, aggiornando automaticamente il valore presente nel database. Tale operazione avviene in maniera completamente trasparente all'utente.

4.7 Decisioni Chiave e Alternative

Durante la progettazione dell'applicazione sono state valutate diverse soluzioni architetturali. Le principali decisioni adottate sono riportate di seguito.

4.7.1 Persistenza dei dati

Per la gestione della persistenza è stato scelto il DBMS relazionale *PostgreSQL*, in alternativa all'utilizzo di file di testo o di oggetti serializzati.

Questa soluzione consente una gestione strutturata delle informazioni e delle relazioni tra le diverse entità del sistema (utenti, ristoranti, recensioni e preferiti), garantendo il rispetto dei

vincoli di integrità referenziale e permettendo di eseguire interrogazioni efficienti mediante il linguaggio SQL.

4.7.2 Interfaccia utente

L'interfaccia grafica è stata sviluppata utilizzando *Java Swing*.

Come possibile alternativa è stato preso in considerazione *JavaFX*, tuttavia Swing è stato preferito poiché è incluso nella piattaforma Java utilizzata per il progetto e non richiede l'installazione o la configurazione di moduli aggiuntivi, semplificando così lo sviluppo e la distribuzione dell'applicazione.

4.7.3 Comunicazione client-server

La comunicazione tra client e server è stata implementata mediante *socket TCP*, attraverso lo scambio di oggetti serializzati di tipo `Richiesta` e `Risposta`.

In alternativa sarebbe stato possibile utilizzare *Java RMI (Remote Method Invocation)*. La soluzione basata sui socket è stata preferita perché offre un controllo più diretto sul protocollo di comunicazione, risulta più semplice da personalizzare e non richiede la configurazione di un registro RMI.

4.7.4 Gestione della concorrenza

Per la gestione delle connessioni concorrenti il server adotta un modello *thread-per-client*: ogni nuova connessione genera un'istanza della classe `ClientHandler`, eseguita in un thread dedicato.

Come alternativa è stato valutato l'impiego di un *thread pool* mediante `ExecutorService`, soluzione generalmente più scalabile. Tuttavia, considerati gli obiettivi didattici del progetto e il numero limitato di client concorrenti previsto, si è preferito il modello a thread dedicati, caratterizzato da una maggiore semplicità implementativa e da una più immediata comprensione del flusso di esecuzione.

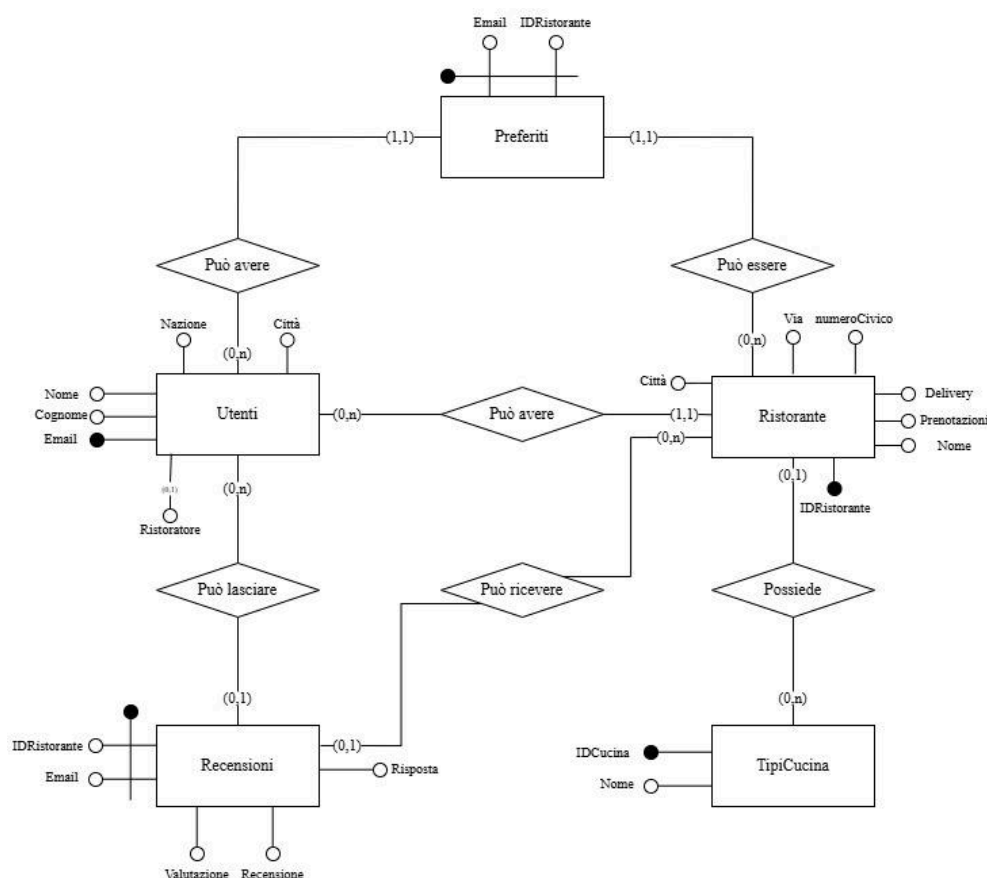
5. Gestione e Persistenza dei Dati

I dati dell'applicazione sono gestiti tramite database relazionale *PostgreSQL*, organizzato in cinque tabelle: *utente*, *ristorante*, *tipicucina*, *recensioni*, *preferiti*.

L'accesso ai dati è mediato dal pattern *DAO (Data Access Object)*: ciascuna entità principale è associata a una classe DAO dedicata (*UtenteDAO*, *RistoranteDAO*, *RecensioneDAO*, *PreferitiDAO*, *TipoCucinaDAO*), responsabile della costruzione ed esecuzione delle query SQL e della conversione dei risultati in oggetti di dominio o DTO.

5.1 Schema del Database

Il database relazionale “*TheKnife*” è composto da cinque tabelle: *Utente*, *TipoCucina*, *Ristorante*, *Preferiti* e *Recensioni*. Lo schema riprende 1:1 la struttura delle entità di dominio descritte nel paragrafo 3.3, con l'aggiunta dei vincoli di integrità referenziale (chiavi esterne) che legano le tabelle tra loro. La Figura 5 riporta lo schema Entità-Relazione (ER) completo, mentre le tabelle seguenti riportano, per ciascuna entità, l'elenco dettagliato dei campi, dei tipi di dato e dei vincoli applicati a livello di DDL PostgreSQL.



5.1.1 Tabella utenti

```
CREATE TABLE Utente(  
    Email VARCHAR(50) PRIMARY KEY,  
    NomeUtente VARCHAR(50) NOT NULL,  
    CognomeUtente VARCHAR(50) NOT NULL,  
    HashPWD VARCHAR(100) NOT NULL,  
    Citta VARCHAR(30) NOT NULL,  
    Nazione VARCHAR(30) NOT NULL,  
    Ristoratore BOOLEAN NOT NULL  
);
```

5.1.2 Tabella TipiCucina

```
CREATE TABLE TipiCucina(  
    IDTipiCucina SERIAL PRIMARY KEY,  
    TipiCucina VARCHAR(30) UNIQUE NOT NULL  
);
```

5.1.3 Tabella Ristorante

```
CREATE TABLE Ristorante(  
    IDRistorante SERIAL PRIMARY KEY,  
    NomeRistorante VARCHAR(50) NOT NULL,  
    Email VARCHAR(50) NOT NULL,  
    Citta VARCHAR(50) NOT NULL,  
    Nazione VARCHAR(50) NOT NULL,  
    Via VARCHAR(50) NOT NULL,  
    NumeroCivico INT NOT NULL,  
    FasciaPrezzo INT NOT NULL,  
    Delivery BOOLEAN NOT NULL,  
    PrenotazioneOnline BOOLEAN NOT NULL,  
    IDTipiCucina INT,  
    FOREIGN KEY (Email)  
        REFERENCES Utente(Email) ON DELETE CASCADE ON UPDATE CASCADE,  
    FOREIGN KEY (IDTipiCucina)  
        REFERENCES TipiCucina(IDTipiCucina) ON DELETE SET NULL ON UPDATE  
CASCADE  
);
```

5.1.4 Tabella Preferiti

```
CREATE TABLE Preferiti(  
    IDRistorante INT,  
    Email VARCHAR(50),  
    PRIMARY KEY (IDRistorante, Email),  
    FOREIGN KEY (IDRistorante)  
        REFERENCES Ristorante(IDRistorante) ON DELETE CASCADE ON UPDATE  
CASCADE,  
    FOREIGN KEY (Email)  
        REFERENCES Utente(Email) ON DELETE CASCADE ON UPDATE CASCADE  
);
```

5.1.5 Tabella Recensioni

```
CREATE TABLE Recensioni(  
    Email VARCHAR(50),  
    IDRistorante INT,  
    Valutazione INT NOT NULL,  
    Recensione VARCHAR(750),  
    Risposta VARCHAR(750),  
    PRIMARY KEY (Email, IDRistorante),  
    FOREIGN KEY (Email)  
        REFERENCES Utente(Email) ON DELETE CASCADE ON UPDATE CASCADE,  
    FOREIGN KEY (IDRistorante)  
        REFERENCES Ristorante(IDRistorante) ON DELETE CASCADE ON UPDATE  
    CASCADE  
);
```

Le politiche di cancellazione a cascata (`ON DELETE CASCADE`) applicate su `Ristorante`, `Preferiti` e `Recensioni` assicurano che l'eliminazione di un utente o di un ristorante comporti automaticamente la rimozione dei record dipendenti, evitando riferimenti orfani. Fa eccezione la chiave esterna tra `Ristorante` e `TipiCucina`, per la quale è stata adottata la politica `ON DELETE SET NULL`: l'eliminazione di un tipo di cucina non comporta la cancellazione dei ristoranti associati, ma solo l'azzeramento del riferimento `IDTipiCucina`.

5.2 Struttura interna DAO

Tutte le classi DAO seguono una struttura uniforme:

- Le query SQL sono definite come *text block* java (“...”) all’interno del metodo che le esegue;
- L’esecuzione avviene tramite *PreparedStatement*, con parametri impostati tramite i metodi `setString/setInt/setBoolean`, evitando la concatenazione diretta di stringhe e proteggendo così da SQL injection;
- La connessione (`DatabaseConnection.getConnection()`), lo *Statement* e l’eventuale *ResultSet* sono dichiarati in un blocco *try-with-resources*, che ne garantisce la chiusura automatica anche in presenza di eccezioni;
- Dove è previsto il recupero di più risultati, ciascuna classe DAO definisce un metodo privato *map(ResultSet)* dedicato alla conversione di una singola riga del *ResultSet* nel corrispondente oggetto Java

5.3 Connessione al database

La classe *DatabaseConnection* centralizza l’accesso al database tramite il metodo statico *getConnection()*, che apre una nuova connessione ad ogni chiamata tramite *DriverManager* (nessun pattern *Singleton* o *connection pooling*): la connessione viene chiusa automaticamente al termine di ogni operazione dal blocco *try-with-resource* che la utilizza. Le credenziali (URL, utente, password) sono lette da variabili d’ambiente (*db_url*, *db_user*, *db_password*; [sez. 2.3](#))

5.4 Gestione delle eccezioni

Il comportamento in caso di errore SQL non è uniforme tra i metodi DAO: alcuni (es. `UtenteDAO.registrazione()`) rilanciano *SQLException* incapsulata in un'eccezione applicativa (*UtenteException*), mentre la maggior parte cattura l'errore localmente, lo stampa su console e restituisce un valore di default (*false* o lista vuota), senza propagarlo al chiamante.

A livello di modello, la classe *Recensione* valida autonomamente i propri invarianti: il costruttore verifica che la valutazione sia compresa tra 1 e 5, sollevando *RecensioniException* in caso contrario.

5.5 Indici

Oltre agli indici generati automaticamente dal DBMS per le chiavi primarie e per gli attributi *UNIQUE*, lo schema attuale non prevede indici aggiuntivi definiti esplicitamente in fase di creazione del database.

In particolare, PostgreSQL genera automaticamente un indice B-tree per ciascuna chiave primaria (`Utente.Email`, `Ristorante.IDRistorante`, `TipiCucina.IDTipoCucina`, la chiave composta (`IDRistorante`, `Email`) di *Preferiti* e la chiave composta (`Email`, `IDRistorante`) di *Recensioni*) e per l'attributo *UNIQUE* `TipiCucina.TipoCucina`.

Le funzionalità che maggiormente beneficerebbero di indici dedicati sono la ricerca dei ristoranti con filtro (`RistoranteDAO.cercaConFiltro()`), che utilizza l'operatore *ILIKE* sui campi `numeratoristorante`, `citta` e `nazione`, e il recupero delle recensioni per ristorante (`RecensioneDAO.getRecensioniByRistorante()`), che filtra sulla colonna `idristorante`, già coperta parzialmente dall'indice implicito della chiave esterna.

Poiché l'operatore *ILIKE* non sfrutta un indice B-tree standard, un eventuale miglioramento futuro potrebbe prevedere l'introduzione di indici *GIN* basati sull'estensione `pg_trgm`, ottimizzati per ricerche parziali case-insensitive su testo.

Considerato il volume di dati previsto per un progetto didattico e le prestazioni comunque adeguate offerte dal motore di PostgreSQL su tabelle di dimensioni contenute, non è stato ritenuto necessario introdurre indici aggiuntivi nella versione attuale del sistema.

5.6 Query SQL

Di seguito vengono riportate le principali query SQL parametriche (*PreparedStatement*) impiegate dalle classi DAO per l'esecuzione delle operazioni CRUD (*Create*, *Read*, *Update*, *Delete*) su ciascuna tabella del database. Le query, raggruppate per entità coerentemente con la struttura del Data Access Layer descritta nella Sezione 5.2, sono definite come *text block* Java (""...""") all'interno del metodo che le esegue, come indicato nella Sezione 5.2.

5.6.1 Utente

Le seguenti query sono definite nella classe `UtenteDAO`:

- **Registrazione:**

```
INSERT INTO utente (  
    email, nomeutente, cognomeutente, hashpwd,  
    citta, nazione, ristoratore  
)  
VALUES (?, ?, ?, ?, ?, ?, ?)
```

- **Ricerca tramite email:**

```
SELECT email, nomeutente, cognomeutente, hashpwd, nazione, citta, ristoratore  
FROM utente  
WHERE email = ?
```

- **Ricerca di tutti gli utenti:**

```
SELECT email, nomeutente, cognomeutente, hashpwd, nazione, citta, ristoratore  
FROM utente
```

- **Controllo esistenza:**

```
SELECT 1  
FROM utente  
WHERE email = ?  
LIMIT 1
```

- **Aggiornamento dell'hash della password:**

```
UPDATE utente  
SET hashpwd = ?  
WHERE email = ?
```

- **Aggiornamento di un singolo campo (nome, cognome, città, nazione, password o ruolo ristoratore), selezionato dinamicamente tramite l'enumerazione `CampoUtente`:**

```
UPDATE utente SET <campo> = ? WHERE email = ?
```

- **Cancellazione:**

```
DELETE FROM utente  
WHERE email = ?
```

5.6.2 Ristorante

Le seguenti query sono definite nella classe `RistoranteDAO`:

- **Inserimento:**

```
INSERT INTO ristorante (  
    numeristorante, email, citta, nazione, via, numeroCivico,  
    fasciaPrezzo, delivery, prenotazioneOnline, idtipocucina  
)  
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
```

- **Ricerca di tutti i ristoranti:**

```
SELECT idristorante, numeristorante, email, citta, nazione, via,  
numeroCivico,  
    fasciaPrezzo, delivery, prenotazioneOnline, idtipocucina  
FROM ristorante  
ORDER BY numeristorante ASC
```

- **Ricerca dei ristoranti di un determinato ristorante:**

```
SELECT idristorante, numeristorante, email, citta, nazione, via,  
numeroCivico,  
    fasciaPrezzo, delivery, prenotazioneOnline, idtipocucina  
FROM ristorante  
WHERE email = ?
```

- **Ricerca con filtro multi-criterio, costruita dinamicamente aggiungendo le clausole AND corrispondenti ai soli parametri effettivamente valorizzati (cfr. Sezione 6.1):**

```
SELECT idristorante, numeristorante, email, citta, nazione, via,  
numeroCivico,  
    fasciaPrezzo, delivery, prenotazioneOnline, idtipocucina  
FROM ristorante  
WHERE 1=1  
    [AND numeristorante ILIKE ?]  
    [AND citta ILIKE ?]  
    [AND nazione ILIKE ?]  
    [AND fasciaprezzo <= ?]  
    [AND delivery = ?]  
    [AND prenotazioneonline = ?]  
    [AND idtipocucina = ?]
```

- **Controllo esistenza, basato sulla coppia nome–e-mail del ristorante:**

```
SELECT 1  
FROM ristorante  
WHERE numeristorante = ? AND email = ?  
LIMIT 1
```

- Aggiornamento di un singolo campo (nome, città, nazione, via o fascia di prezzo), identificato tramite `idristorante` e selezionato dinamicamente tramite l'enumerazione `CampoRistorante`:

```
UPDATE ristorante SET <campo> = ? WHERE idristorante = ?
```

- Aggiornamento completo di tutti i campi:

```
UPDATE ristorante
SET nomeRistorante = ?,
    città = ?,
    nazione = ?,
    via = ?,
    numeroCivico = ?,
    fasciaPrezzo = ?,
    delivery = ?,
    prenotazioneOnline = ?,
    idtipoCucina = ?
WHERE idristorante = ?
```

- Cancellazione:

```
DELETE FROM ristorante WHERE idristorante = ?
```

5.6.3 TipiCucina

La seguente query è definita nella classe `TipoCucinaDAO`:

- Ricerca di tutti i tipi di cucina:

```
SELECT idtipocucina, tipocucina
FROM tipicucina
```

La tabella `tipicucina` è popolata staticamente in fase di creazione del database e non prevede, allo stato attuale, operazioni di inserimento, aggiornamento o cancellazione lato applicativo.

5.6.4 Preferiti

Le seguenti query sono definite nella classe `PreferitiDAO`:

- Inserimento:

```
INSERT INTO preferiti (email, idristorante)
VALUES (?, ?)
```

- Controllo esistenza:

```
SELECT 1
FROM preferiti
WHERE email = ? AND idristorante = ?
```

LIMIT 1

- Ricerca dei preferiti di un utente:

```
SELECT email, idristorante  
FROM preferiti  
WHERE email = ?
```

- Cancellazione:

```
DELETE FROM preferiti  
WHERE email = ? AND idristorante = ?
```

5.6.5 Recensioni

Le seguenti query sono definite nella classe `RecensioneDAO`:

- Inserimento:

```
INSERT INTO recensioni (email, idristorante, valutazione, recensione,  
risposta)  
VALUES (?, ?, ?, ?, ?)
```

- Ricerca di tutte le recensioni:

```
SELECT email, idristorante, valutazione, recensione, risposta  
FROM recensioni
```

- Ricerca delle recensioni di un ristorante:

```
SELECT email, idristorante, valutazione, recensione, risposta  
FROM recensioni  
WHERE idristorante = ?  
ORDER BY email DESC
```

- Ricerca delle recensioni di un utente:

```
SELECT email, idristorante, valutazione, recensione, risposta  
FROM recensioni  
WHERE email = ?
```

- Riepilogo (numero totale e valutazione media) delle recensioni di un ristorante, calcolato lato DBMS tramite le funzioni di aggregazione `COUNT()` e `AVG()` (cfr. Sezione 6.4):

```
SELECT COUNT(*) AS totRecensioni,  
COALESCE(AVG(valutazione), 0) AS mediaValutazione  
FROM recensioni  
WHERE idristorante = ?
```

- Aggiornamento di un singolo campo (recensione, valutazione o risposta), selezionato dinamicamente tramite l'enumerazione `CampoRecensione`:

```
UPDATE recensioni SET <campo> = ? WHERE idristorante = ? AND email = ?
```

- Risposta a una recensione da parte del ristorante, consentita solo in assenza di una risposta già presente:

```
UPDATE recensioni
SET risposta = ?
WHERE idristorante = ?
AND email = ?
AND (risposta IS NULL OR risposta = '')
```

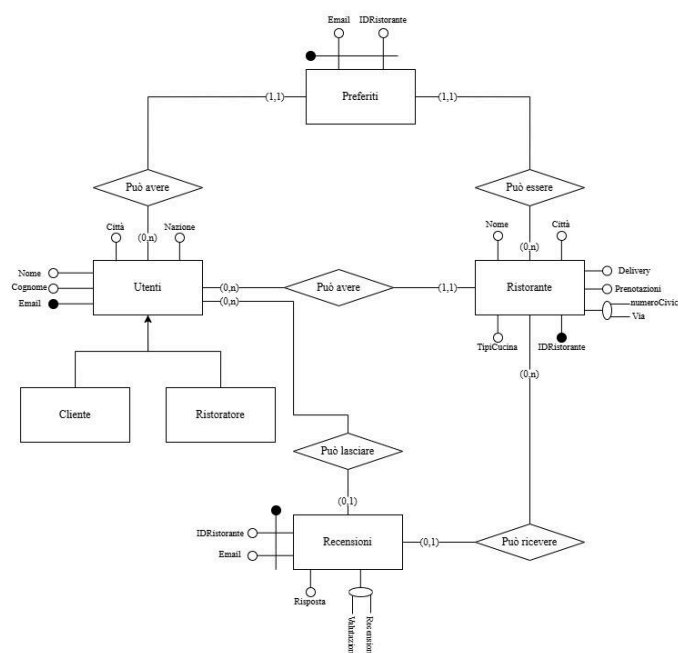
- Cancellazione:

```
DELETE FROM recensioni
WHERE idristorante = ? AND email = ?
```

5.7 Schema relazionale

- Utenti (Email, HashPwd, NomeUtente, CognomeUtente, Città, Nazione Ristoratore)
- TipiCucina (IDTipoCucina, TipoCucina)
- Ristoranti (IDRistorante, NomeRistorante, Email^{UTENTE}, Città, Nazione, Via, NumeroCivico, FasciaPrezzo, Delivery, PrenotazioniOnline, IDTipoCucina^{TIPICUCINA})
- Recensioni (Email^{UTENTE}, IDRistorante^{RISTORANTE}, Valutazione, Recensione, Risposta)
- Preferiti (IDRistorante^{RISTORANTE}, Email^{UTENTE})

5.8 Schema E-R non ristrutturato



6. Scelte algoritmiche

6.1 Ricerca dei ristoranti

La funzionalità di ricerca dei ristoranti è implementata dal metodo `RistoranteDAO.cercaConFiltro()`, che costruisce dinamicamente la query SQL in funzione dei criteri di ricerca specificati dall'utente.

La clausola `WHERE` viene inizializzata con la condizione sempre vera `WHERE 1 = 1`, alla quale vengono aggiunte progressivamente le clausole `AND` corrispondenti ai filtri effettivamente valorizzati, quali nome del ristorante, città, nazione, fascia di prezzo massima, disponibilità del servizio di delivery, prenotazione online e tipologia di cucina.

Per i campi testuali (nome, città e nazione) viene utilizzato l'operatore `ILIKE` con il pattern `%valore%`, che consente di effettuare ricerche parziali senza distinzione tra lettere maiuscole e minuscole.

Tutti i valori sono passati alla query tramite un `PreparedStatement`, evitando così possibili vulnerabilità di tipo SQL Injection, anche in presenza di un numero variabile di filtri.

Questa soluzione consente di demandare l'elaborazione della ricerca direttamente al DBMS, evitando il caricamento in memoria dell'intera tabella dei ristoranti e sfruttando le ottimizzazioni offerte dal motore di PostgreSQL.

6.2 Gestione degli utenti e autenticazione

L'autenticazione degli utenti si basa sull'indirizzo e-mail, che costituisce la chiave primaria della tabella `utente` e rappresenta quindi l'identificatore univoco utilizzato durante il login.

Il recupero dell'utente è effettuato dal metodo `UtenteDAO.trovaUtente()` mediante una query del tipo:

```
WHERE email = ?
```

Tale ricerca beneficia dell'indice creato automaticamente da PostgreSQL sulla chiave primaria, garantendo tempi di accesso molto ridotti.

La verifica della password è affidata alla classe `PasswordService`, che confronta l'hash memorizzato nel database con quello ottenuto dalla password inserita dall'utente, secondo il meccanismo descritto nella [Sezione 4.6](#).

Il server è stato progettato come *stateless* rispetto all'autenticazione: la classe `ClientHandler` non mantiene alcuna informazione relativa all'utente autenticato. È quindi il client a conservare localmente l'identità dell'utente (indirizzo e-mail) e a trasmetterla nelle richieste successive che richiedono autorizzazione, come avviene, ad esempio, nella richiesta `GET_UTENTE` inviata dopo un'autenticazione completata con successo (Figura 11).

6.3 Inserimento e gestione dei ristoranti

L'inserimento di un nuovo ristorante è gestito dal metodo

`RistoranteService.creaRistorante()`, che esegue una serie di controlli preliminari sui dati forniti.

I campi testuali (nome, città, nazione, via e numero civico) vengono validati mediante la classe `ValidationUtils`, mentre il metodo `RistoranteDAO.esisteRistorante()` verifica che il ristorante non abbia già registrato un locale con lo stesso nome. Il controllo di unicità è basato sulla coppia *nome–e-mail del ristorante*.

Attualmente tale verifica è implementata esclusivamente a livello applicativo. Lo schema del database non prevede infatti un vincolo `UNIQUE` sulla coppia (nome, email), pertanto non viene impedito l'inserimento di duplicati nel caso di richieste concorrenti provenienti dallo stesso ristorante.

Il recupero di un ristorante tramite identificativo è realizzato dal metodo

`RistoranteService.getRistorante()`. Attualmente non è disponibile una query dedicata nel livello DAO: viene recuperato l'intero elenco dei ristoranti mediante `getTuttiRistoranti()` e successivamente filtrato in memoria per individuare il record con l'identificativo richiesto.

Questa soluzione, adottata principalmente per la costruzione degli oggetti `RistoranteDTO` associati ai preferiti, risulta semplice da implementare ma poco efficiente, poiché richiede il trasferimento dell'intera tabella anche quando è necessario recuperare un solo ristorante.

6.4 Recensioni e punteggi

Il calcolo del numero totale di recensioni e della valutazione media di ciascun ristorante è demandato direttamente al DBMS mediante il metodo `RecensioneDAO.getRiepilogo()`.

A tal fine vengono utilizzate le funzioni di aggregazione SQL `COUNT()` e `AVG()`, consentendo al database di eseguire il calcolo senza trasferire in memoria tutte le recensioni associate al ristorante.

Questa scelta è coerente con l'approccio adottato per la ricerca dei ristoranti, privilegiando l'elaborazione lato database e riducendo il volume di dati trasferiti tra il DBMS e l'applicazione.

7. Ulteriori Diagrammi UML

Il presente capitolo raccoglie ulteriori diagrammi UML prodotti durante la progettazione, a integrazione di quelli già presentati nei capitoli precedenti (coerentemente con quanto indicato nel paragrafo 1.1). Per una migliore leggibilità, si raccomanda la consultazione delle versioni ad alta risoluzione disponibili nella cartella UML/ allegata alla documentazione tecnica del progetto.

7.1 Diagrammi delle classi – modulo server

I diagrammi seguenti dettagliano la struttura interna dei package del modulo server, già introdotti a livello di organizzazione dei package in Figura 2.

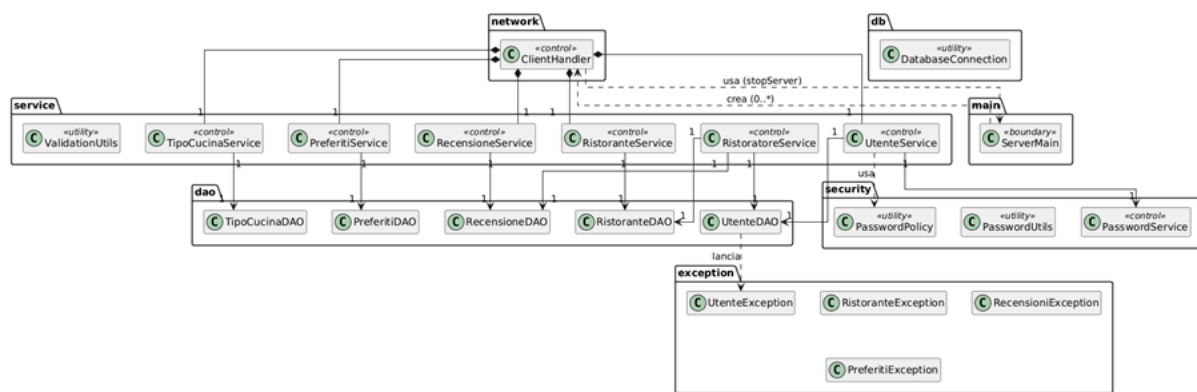


Figura 6: Diagramma delle classi – package server (overview).

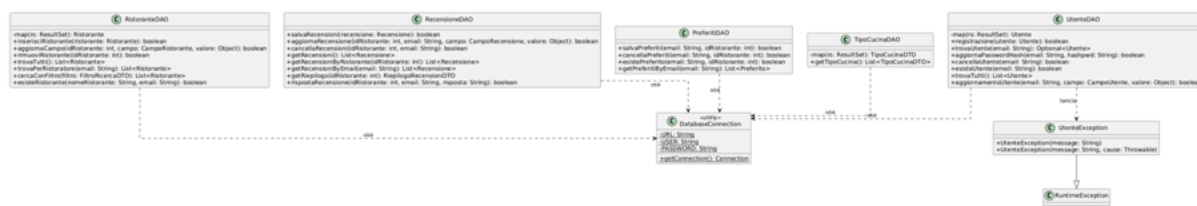


Figura 7: Diagramma delle classi – package server.dao.

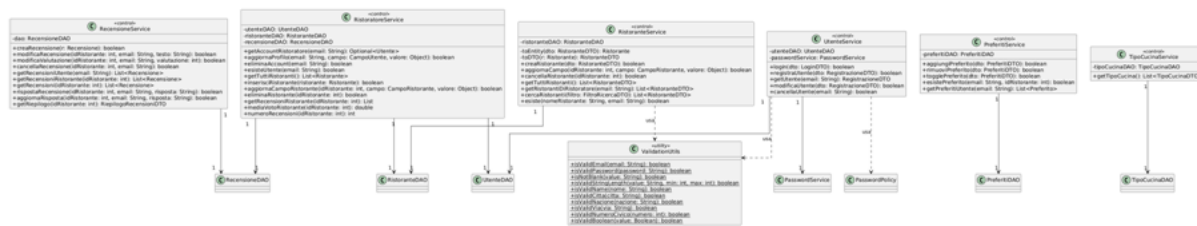


Figura 8: Diagramma delle classi – package server.service.

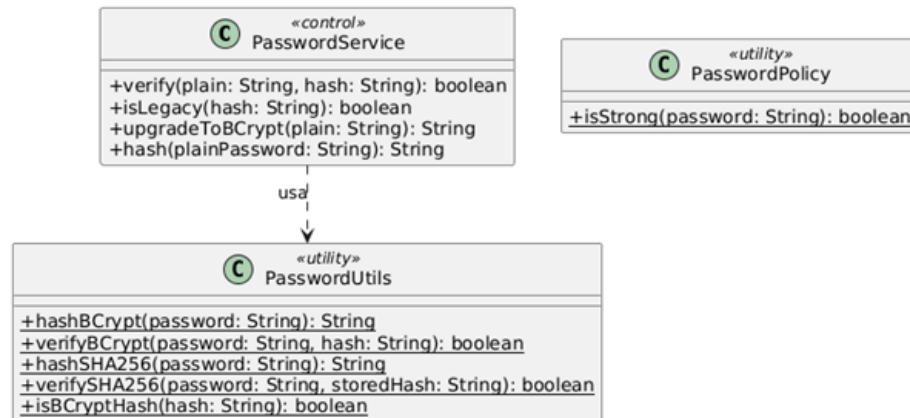


Figura 9: Diagramma delle classi – package `server.security`.

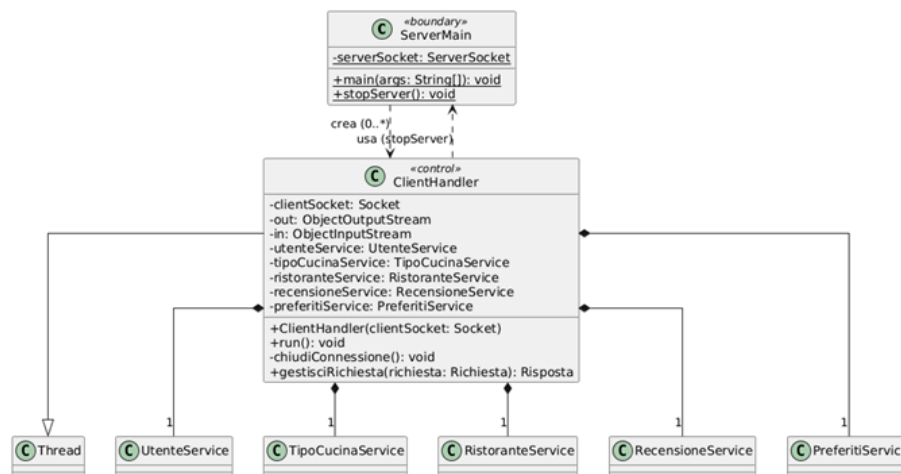


Figura 10: Diagramma delle classi – package `server.network` e `server.main`.

7.2 Diagrammi di sequenza

I diagrammi di sequenza seguenti illustrano lo scambio di messaggi tra client e server per due casi d'uso rappresentativi del protocollo applicativo descritto nel paragrafo 4.3: login e registrazione.

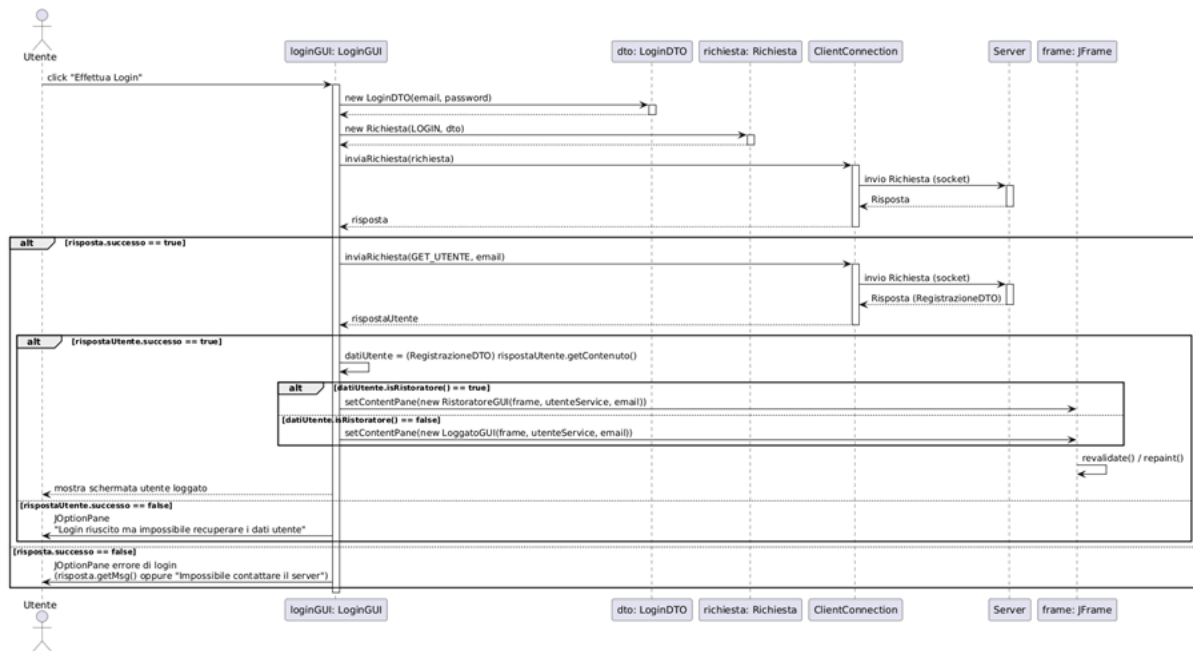


Figura 11: Diagramma di sequenza – Login.

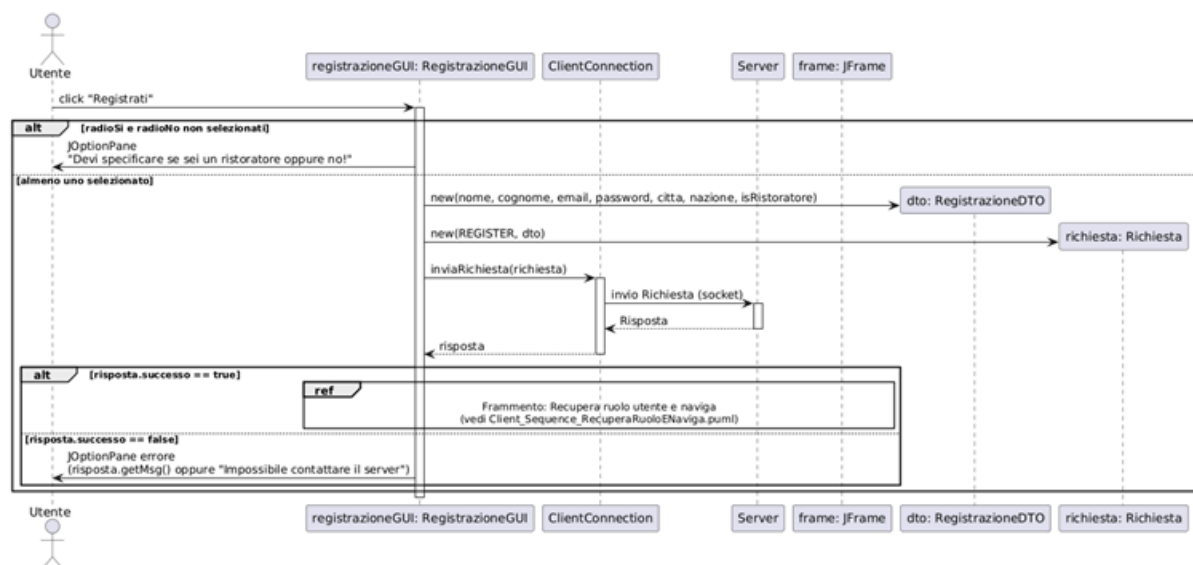


Figura 12: Diagramma di sequenza – Registrazione.

7.3 Diagrammi di stato

I diagrammi di stato seguenti descrivono il ciclo di vita di un thread `ClientHandler` lato server e le transizioni di stato di una `Recensione` lato dominio.

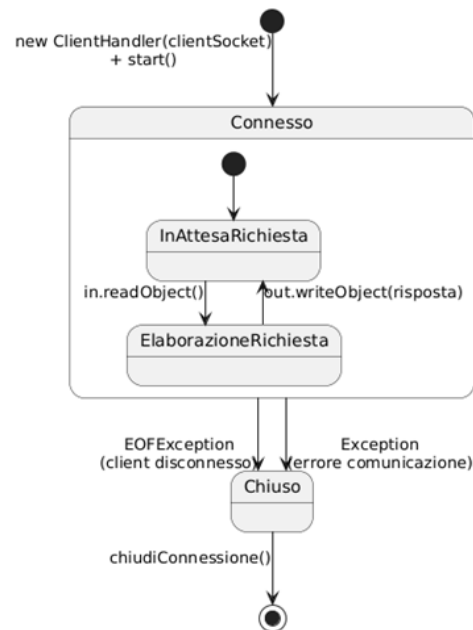


Figura 13: Diagramma di stato – `ClientHandler`.

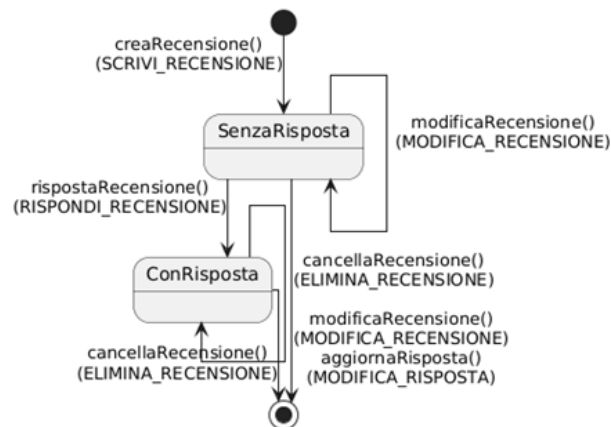


Figura 14: Diagramma di stato – `Recensione`.

7.4 Diagrammi delle classi – modulo client

A completamento della struttura dei package client riportata in Figura 2, si riportano i diagrammi delle classi dei principali package dell'interfaccia grafica.



Figura 15: Diagramma delle classi – package client (overview).



Figura 16: Diagramma delle classi – package client.gui.

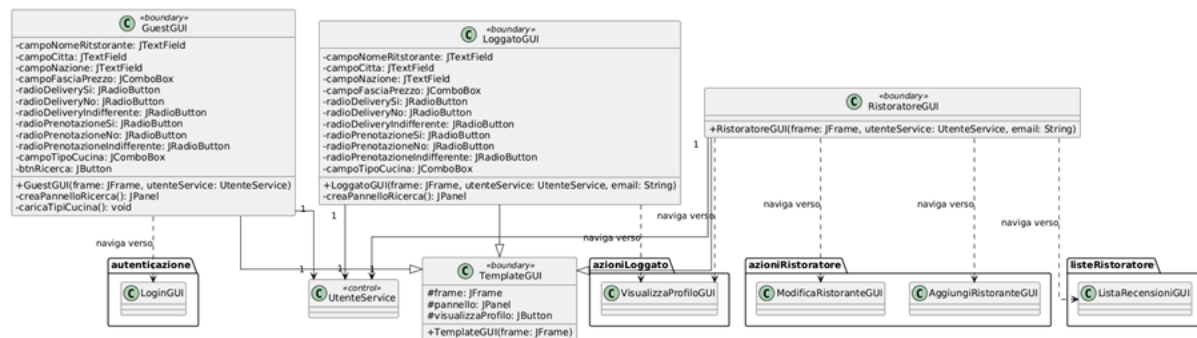


Figura 17: Diagramma delle classi – package client.gui.menu.

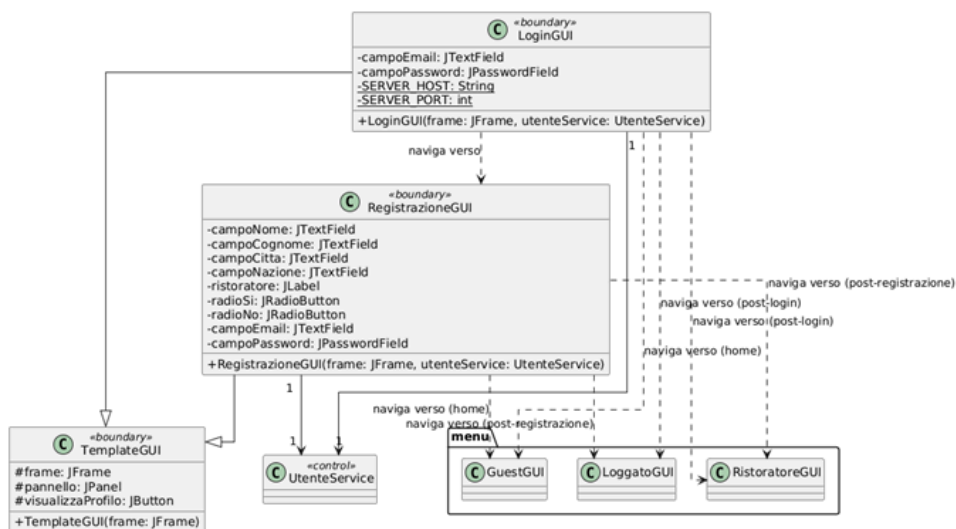
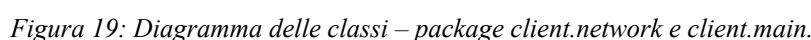


Figura 18: Diagramma delle classi – package client.gui.autenticazione.



Il diagramma seguente riporta la vista completa delle classi del modulo shared, a complemento del dettaglio sulle sole entità di dominio già riportato in Figura 3.



8. Bibliografia e Sitografia

Documentazione Java

[1] **Oracle Java Documentation**, Oracle. Disponibile su: <https://docs.oracle.com/en/java/>.

Descrizione: Documentazione ufficiale Java per lo sviluppo.

[2] **Java Platform, Standard Edition Documentation**, Oracle. Disponibile su: <https://docs.oracle.com/javase/>.

Descrizione: Riferimento per le API Java utilizzate, incluse `java.sql`, java.net, java.io e `javax.swing`.

[3] **Git Repository del Progetto**, GitHub. Disponibile su: <https://github.com/Cattnk04/TheKnifeLabB.git>.

Descrizione: Repository contenente il codice sorgente dell'applicazione

Documentazione database e librerie esterne

[4] **PostgreSQL Documentation**, PostgreSQL Global Development Group. Disponibile su: <https://www.postgresql.org/docs/>

[5] **PostgreSQL JDBC Driver (pgJDBC) Documentation**. Disponibile su: <https://jdbc.postgresql.org/>

Strumenti di Sviluppo

[6] **IntelliJ IDEA**, JetBrains. Disponibile su: <https://www.jetbrains.com/idea/>.

Descrizione: IDE principale utilizzato per lo sviluppo Java.

[7] **PlantText**. Disponibile su: <https://www.planttext.com/>

Descrizione: Strumento online utilizzato per la realizzazione dei diagrammi UML in linguaggio PlantUML.